roots.h

```
template <class T>
Doub rtbis(T &func, const Doub x1, const Doub x2, const Doub xacc) {
```
Using bisection, return the root of a function or functor `func` known to lie between `x1` and `x2`. The root will be refined until its accuracy is ±`xacc`.
```
    const Int JMAX=50;              Maximum allowed number of bisections.
    Doub dx,xmid,rtb;
    Doub f=func(x1);
    Doub fmid=func(x2);
    if (f*fmid >= 0.0) throw("Root must be bracketed for bisection in rtbis");
    rtb = f < 0.0 ? (dx=x2-x1,x1) : (dx=x1-x2,x2);      Orient the search so that f>0
    for (Int j=0;j<JMAX;j++) {                              lies at x+dx.
        fmid=func(xmid=rtb+(dx *= 0.5));                    Bisection loop.
        if (fmid <= 0.0) rtb=xmid;
        if (abs(dx) < xacc || fmid == 0.0) return rtb;
    }
    throw("Too many bisections in rtbis");
}
```

9.2 Secant Method, False Position Method, and Ridders' Method

For functions that are smooth near a root, the methods known respectively as *false position* (or *regula falsi*) and the *secant method* generally converge faster than bisection. In both of these methods the function is assumed to be approximately linear in the local region of interest, and the next improvement in the root is taken as the point where the approximating line crosses the axis. After each iteration, one of the previous boundary points is discarded in favor of the latest estimate of the root.

The *only* difference between the methods is that secant retains the most recent of the prior estimates (Figure 9.2.1; this requires an arbitrary choice on the first iteration), while false position retains that prior estimate for which the function value has the opposite sign from the function value at the current best estimate of the root, so that the two points continue to bracket the root (Figure 9.2.2). Mathematically, the secant method converges more rapidly near a root of a sufficiently continuous function. Its order of convergence can be shown to be the "golden ratio" 1.618..., so that

$$\lim_{k\to\infty} |\epsilon_{k+1}| \approx \text{const} \times |\epsilon_k|^{1.618} \tag{9.2.1}$$

The secant method has, however, the disadvantage that the root does not necessarily remain bracketed. For functions that are *not* sufficiently continuous, the algorithm can therefore not be guaranteed to converge: Local behavior might send it off toward infinity.

False position, since it sometimes keeps an older rather than newer function evaluation, has a lower order of convergence. Since the newer function value will *sometimes* be kept, the method is often superlinear, but estimation of its exact order is not so easy.

Here are sample implementations of these two related methods. While these methods are standard textbook fare, *Ridders' method*, described below, or *Brent's method*, described in the next section, are almost always better choices. Figure 9.2.3 shows the behavior of the secant and false-position methods in a difficult situation.

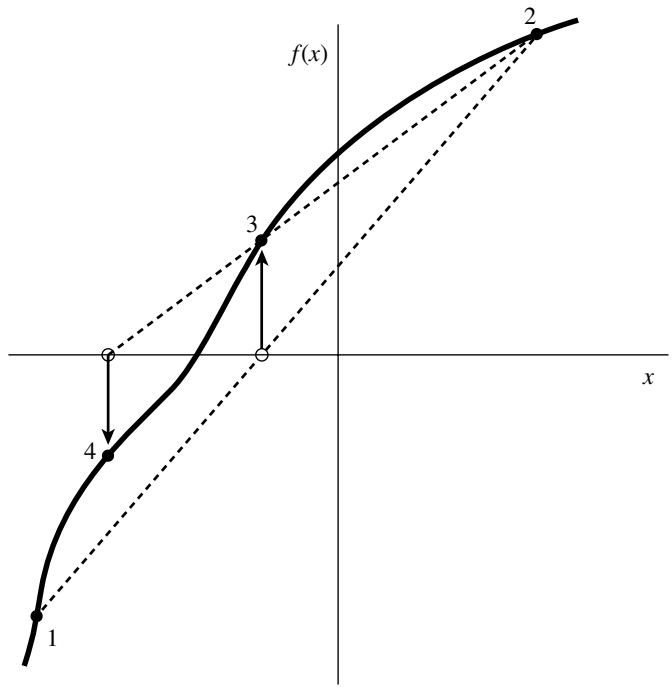

Figure 9.2.1. Secant method. Extrapolation or interpolation lines (dashed) are drawn through the two most recently evaluated points, whether or not they bracket the function. The points are numbered in the order that they are used.

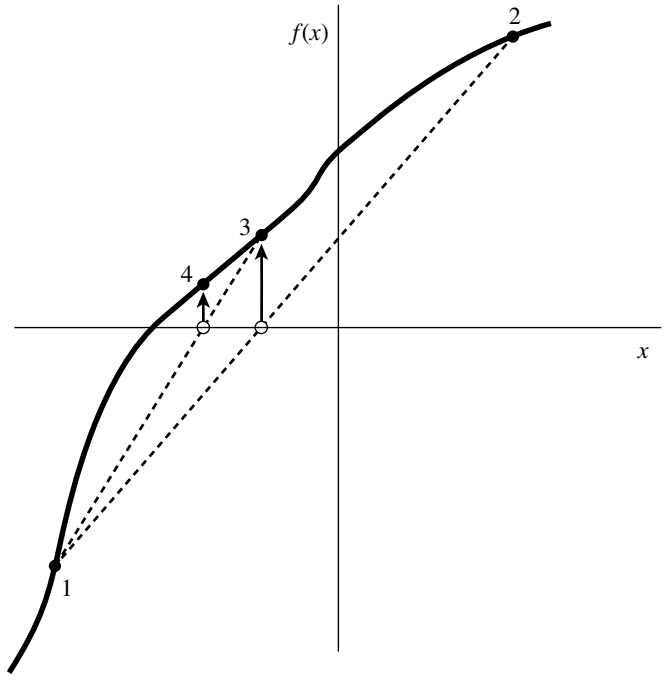

Figure 9.2.2. False-position method. Interpolation lines (dashed) are drawn through the most recent points *that bracket the root*. In this example, point 1 thus remains "active" for many steps. False position converges less rapidly than the secant method, but it is more certain.

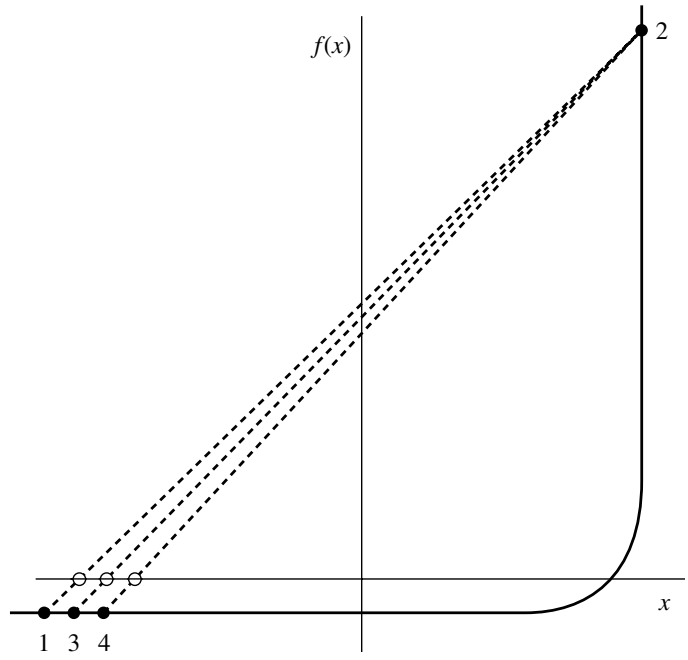

Figure 9.2.3. Example where both the secant and false-position methods will take many iterations to arrive at the true root. This function would be difficult for many other root-finding methods.

roots.h

```
template <class T>
Doub rtflsp(T &func, const Doub x1, const Doub x2, const Doub xacc) {
```

Using the false-position method, return the root of a function or functor `func` known to lie between `x1` and `x2`. The root is refined until its accuracy is ±`xacc`.

```
    const Int MAXIT=30;                 Set to the maximum allowed number of iterations.
    Doub xl,xh,del;
    Doub fl=func(x1);
    Doub fh=func(x2);                   Be sure the interval brackets a root.
    if (fl*fh > 0.0) throw("Root must be bracketed in rtflsp");
    if (fl < 0.0) {                     Identify the limits so that xl corresponds to the low
        xl=x1;                              side.
        xh=x2;
    } else {
        xl=x2;
        xh=x1;
        SWAP(fl,fh);
    }
    Doub dx=xh-xl;
    for (Int j=0;j<MAXIT;j++) {         False-position loop.
        Doub rtf=xl+dx*fl/(fl-fh);      Increment with respect to latest value.
        Doub f=func(rtf);
        if (f < 0.0) {                  Replace appropriate limit.
            del=xl-rtf;
            xl=rtf;
            fl=f;
        } else {
            del=xh-rtf;
            xh=rtf;
            fh=f;
        }
        dx=xh-xl;
        if (abs(del) < xacc || f == 0.0) return rtf;      Convergence.
```

```
    }
    throw("Maximum number of iterations exceeded in rtflsp");
}
```

roots.h

```
template <class T>
Doub rtsec(T &func, const Doub x1, const Doub x2, const Doub xacc) {
```

Using the secant method, return the root of a function or functor `func` thought to lie between `x1` and `x2`. The root is refined until its accuracy is ±`xacc`.

```
    const Int MAXIT=30;                 Maximum allowed number of iterations.
    Doub xl,rts;
    Doub fl=func(x1);
    Doub f=func(x2);
    if (abs(fl) < abs(f)) {             Pick the bound with the smaller function value as
        rts=x1;                             the most recent guess.
        xl=x2;
        SWAP(fl,f);
    } else {
        xl=x1;
        rts=x2;
    }
    for (Int j=0;j<MAXIT;j++) {         Secant loop.
        Doub dx=(xl-rts)*f/(f-fl);      Increment with respect to latest value.
        xl=rts;
        fl=f;
        rts += dx;
        f=func(rts);
        if (abs(dx) < xacc || f == 0.0) return rts;      Convergence.
    }
    throw("Maximum number of iterations exceeded in rtsec");
}
```

9.2.1 Ridders' Method

A powerful variant on false position is due to Ridders [1]. When a root is bracketed between x_1 and x_2, Ridders' method first evaluates the function at the midpoint $x_3 = (x_1 + x_2)/2$. It then factors out that unique exponential function that turns the residual function into a straight line. Specifically, it solves for a factor e^Q that gives

$$f(x_1) - 2f(x_3)e^Q + f(x_2)e^{2Q} = 0 \tag{9.2.2}$$

This is a quadratic equation in e^Q, which can be solved to give

$$e^Q = \frac{f(x_3) + \text{sign}[f(x_2)]\sqrt{f(x_3)^2 - f(x_1)f(x_2)}}{f(x_2)} \tag{9.2.3}$$

Now the false-position method is applied, not to the values $f(x_1)$, $f(x_3)$, $f(x_2)$, but to the values $f(x_1)$, $f(x_3)e^Q$, $f(x_2)e^{2Q}$, yielding a new guess for the root, x_4. The overall updating formula (incorporating the solution 9.2.3) is

$$x_4 = x_3 + (x_3 - x_1)\frac{\text{sign}[f(x_1) - f(x_2)]f(x_3)}{\sqrt{f(x_3)^2 - f(x_1)f(x_2)}} \tag{9.2.4}$$

Equation (9.2.4) has some very nice properties. First, x_4 is guaranteed to lie in the interval (x_1, x_2), so the method never jumps out of its brackets. Second, the convergence of successive applications of equation (9.2.4) is *quadratic*, that is, $m = 2$